- AKANSHA BALHARA
- 1/23/SET/BCS/433
- 6AIMLB1
- DATE: 05/03/2026
- EXPERIMENT: 07

House Price Prediction

Training/Testing 80/20

```
# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

# Load Dataset
data = pd.read_csv("/content/data.csv")

print(data.head())
print(data.columns)

# Drop unnecessary columns
data = data.drop(['date','street','city','statezip','country'],
axis=1)

# Define X and Y
X = data.drop('price', axis=1)
y = data['price']

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Normalization
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# -------------------------------
# Main Model Training
# -------------------------------

model = Sequential()
```

```
model.add(Dense(128, activation='relu',
input_shape=(X_train.shape[1],)))
model.add(Dense(64, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(1))

optimizer = Adam(learning_rate=0.001)

model.compile(
    optimizer=optimizer,
    loss='mean_squared_error',
    metrics=['mae']
)

# Model Summary
model.summary()

# Train Model
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2
)

# Evaluate Model
loss, mae = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test MAE:", mae)

# -------------------------------
# Visualization 1: Loss vs Epoch
# -------------------------------

plt.figure()
plt.plot(history.history['loss'], label="Training Loss")
plt.plot(history.history['val_loss'], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss vs Epoch")
plt.legend()
plt.show()

# -------------------------------
# Visualization 2: MAE vs Epoch
# -------------------------------

plt.figure()
```

```
plt.plot(history.history['mae'], label="Training MAE")
plt.plot(history.history['val_mae'], label="Validation MAE")
plt.xlabel("Epoch")
plt.ylabel("MAE")
plt.title("MAE vs Epoch")
plt.legend()
plt.show()

# -------------------------------
# Learning Rate Experiment
# -------------------------------

learning_rates = [0.2, 0.003, 0.04, 0.9]

plt.figure()

for lr in learning_rates:

    temp_model = Sequential()
    temp_model.add(Dense(128, activation='relu',
input_shape=(X_train.shape[1],)))
    temp_model.add(Dense(64, activation='relu'))
    temp_model.add(Dense(32, activation='relu'))
    temp_model.add(Dense(1))

    optimizer = Adam(learning_rate=lr)

    temp_model.compile(
        optimizer=optimizer,
        loss='mean_squared_error',
        metrics=['mae']
    )

    history_lr = temp_model.fit(
        X_train,
        y_train,
        epochs=20,
        batch_size=32,
        verbose=0
    )

    plt.plot(history_lr.history['loss'], label=f"LR = {lr}")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss vs Epoch for Different Learning Rates")
plt.legend()
plt.show()

# -------------------------------
```

```
# MAE vs Epoch for Learning Rates
# -------------------------------

plt.figure()

for lr in learning_rates:

    temp_model = Sequential()
    temp_model.add(Dense(128, activation='relu',
input_shape=(X_train.shape[1],)))
    temp_model.add(Dense(64, activation='relu'))
    temp_model.add(Dense(32, activation='relu'))
    temp_model.add(Dense(1))

    optimizer = Adam(learning_rate=lr)

    temp_model.compile(
        optimizer=optimizer,
        loss='mean_squared_error',
        metrics=['mae']
    )

    history_lr = temp_model.fit(
        X_train,
        y_train,
        epochs=20,
        batch_size=32,
        verbose=0
    )

    plt.plot(history_lr.history['mae'], label=f"LR = {lr}")

plt.xlabel("Epoch")
plt.ylabel("MAE")
plt.title("MAE vs Epoch for Different Learning Rates")
plt.legend()
plt.show()
```

```
                   date      price  bedrooms  bathrooms  sqft_living
sqft_lot  \
0  2014-05-02 00:00:00   313000.0       3.0       1.50         1340
7912
1  2014-05-02 00:00:00  2384000.0       5.0       2.50         3650
9050
2  2014-05-02 00:00:00   342000.0       3.0       2.00         1930
11947
3  2014-05-02 00:00:00   420000.0       3.0       2.25         2000
8030
4  2014-05-02 00:00:00   550000.0       4.0       2.50         1940
10500
```

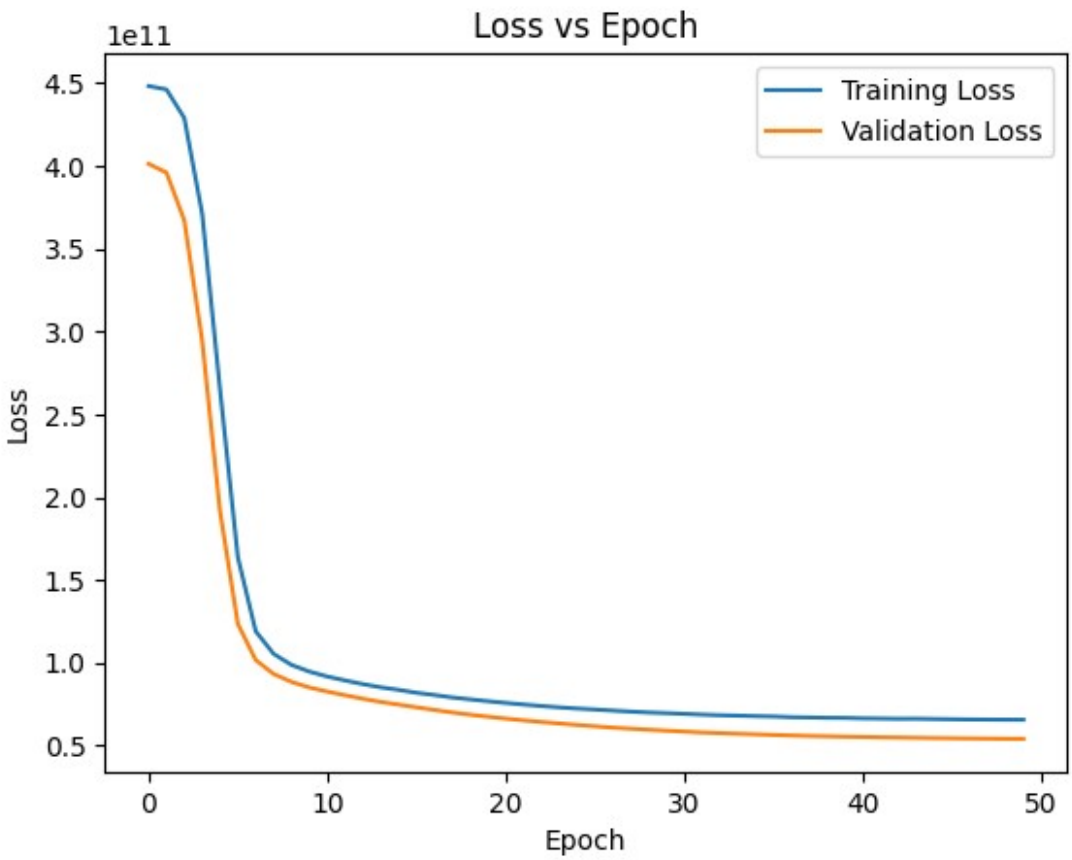
```
   floors  waterfront  view  condition  sqft_above  sqft_basement
yr_built  \
0     1.5           0     0          3        1340              0
1955
1     2.0           0     4          5        3370            280
1921
2     1.0           0     0          4        1930              0
1966
3     1.0           0     0          4        1000           1000
1963
4     1.0           0     0          4        1140            800
1976

   yr_renovated                     street       city  statezip country

0          2005      18810 Densmore Ave N  Shoreline  WA 98133     USA

1             0           709 W Blaine St    Seattle  WA 98119     USA

2             0  26206-26214 143rd Ave SE       Kent  WA 98042     USA

3             0           857 170th Pl NE   Bellevue  WA 98008     USA

4          1992         9105 170th Ave NE    Redmond  WA 98052     USA

Index(['date', 'price', 'bedrooms', 'bathrooms', 'sqft_living',
'sqft_lot',
       'floors', 'waterfront', 'view', 'condition', 'sqft_above',
       'sqft_basement', 'yr_built', 'yr_renovated', 'street', 'city',
       'statezip', 'country'],
      dtype='object')

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)

Model: "sequential"

┏━━━
```

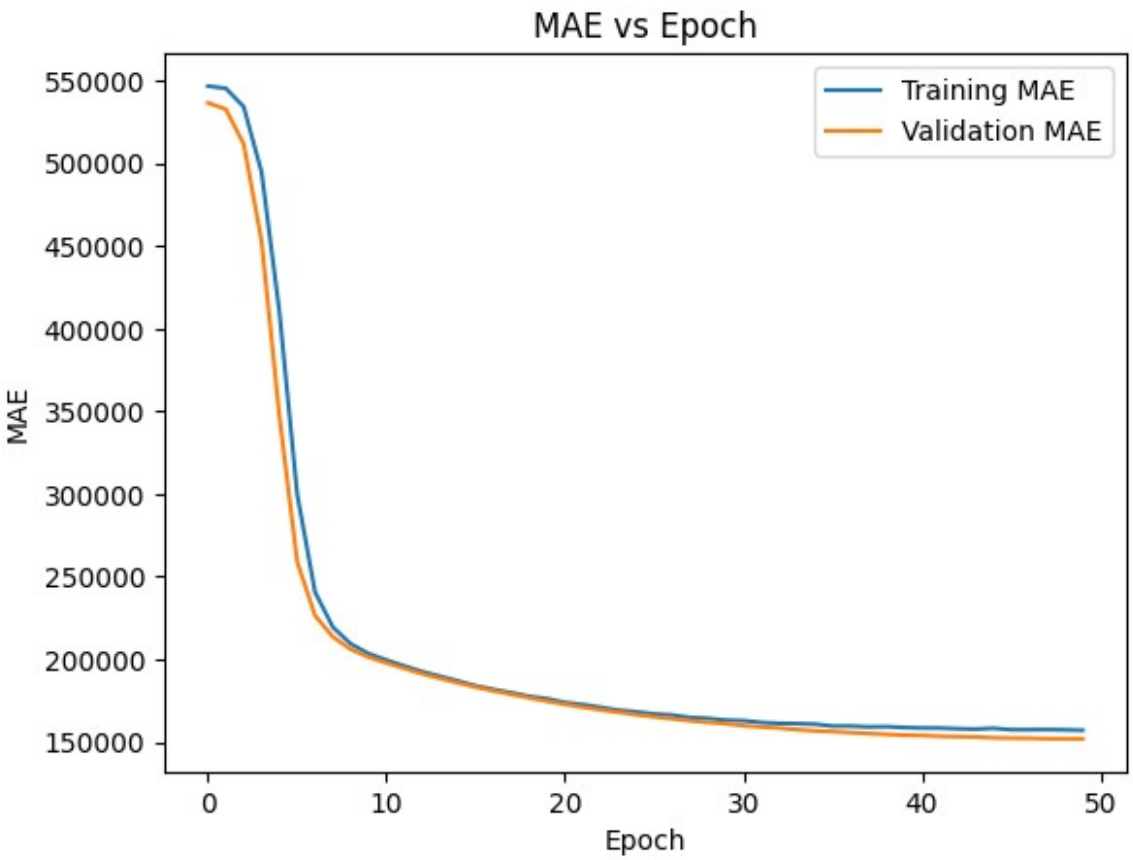
```
──────┤
│ dense_1 (Dense)                  │ (None, 64)              │
8,256 │
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
│ dense_2 (Dense)                  │ (None, 32)              │
2,080 │
├──────────────────────────────────┼─────────────────────────┼──────────
──────┤
│ dense_3 (Dense)                  │ (None, 1)               │
33 │
└──────────────────────────────────┴─────────────────────────┴──────────
──────┘

 Total params: 12,033 (47.00 KB)

 Trainable params: 12,033 (47.00 KB)

 Non-trainable params: 0 (0.00 B)

Epoch 1/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - loss: 523710169088.0000 -
mae: 557832.5000 - val_loss: 401266147328.0000 - val_mae: 536628.4375
Epoch 2/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 470402007040.0000 -
mae: 545210.1875 - val_loss: 395973197824.0000 - val_mae: 532819.0625
Epoch 3/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 437348171776.0000 -
mae: 543775.6875 - val_loss: 367114059776.0000 - val_mae: 511811.2188
Epoch 4/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 380750004224.0000 -
mae: 504221.3750 - val_loss: 294453608448.0000 - val_mae: 453125.2500
Epoch 5/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 275303825408.0000 -
mae: 427898.5625 - val_loss: 191518146560.0000 - val_mae: 347309.8125
Epoch 6/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 169727721472.0000 -
mae: 315125.6875 - val_loss: 123426250752.0000 - val_mae: 258928.8281
Epoch 7/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 120643469312.0000 -
mae: 250167.6094 - val_loss: 101629353984.0000 - val_mae: 226460.4844
Epoch 8/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 120345747456.0000 -
mae: 229423.2812 - val_loss: 93234462720.0000 - val_mae: 213643.7344
Epoch 9/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 93496991744.0000 - mae:
209955.4844 - val_loss: 88444542976.0000 - val_mae: 206111.0469
Epoch 10/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 88880103424.0000 - mae:
201906.6562 - val_loss: 85069520896.0000 - val_mae: 201361.3281
```

```
Epoch 11/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 104127258624.0000 -
mae: 204228.1094 - val_loss: 82607546368.0000 - val_mae: 197869.9844
Epoch 12/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 91566391296.0000 - mae:
197019.0312 - val_loss: 80401776640.0000 - val_mae: 194525.5000
Epoch 13/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 84311924736.0000 - mae:
194091.0156 - val_loss: 78256513024.0000 - val_mae: 191285.2188
Epoch 14/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 78297120768.0000 - mae:
187876.4375 - val_loss: 76360122368.0000 - val_mae: 188496.5625
Epoch 15/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 85319442432.0000 - mae:
190493.3906 - val_loss: 74722975744.0000 - val_mae: 185797.1719
Epoch 16/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 74005192704.0000 - mae:
182280.5156 - val_loss: 73090220032.0000 - val_mae: 183253.2812
Epoch 17/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 69613076480.0000 - mae:
175840.9688 - val_loss: 71499448320.0000 - val_mae: 180946.4219
Epoch 18/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 74939998208.0000 - mae:
174578.5781 - val_loss: 69985050624.0000 - val_mae: 178866.7812
Epoch 19/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 81108451328.0000 - mae:
178683.2500 - val_loss: 68639105024.0000 - val_mae: 176751.5156
Epoch 20/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 85418328064.0000 - mae:
181702.3438 - val_loss: 67398492160.0000 - val_mae: 174671.9531
Epoch 21/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 71458324480.0000 - mae:
170729.7031 - val_loss: 66296926208.0000 - val_mae: 172858.1562
Epoch 22/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 84890673152.0000 - mae:
177179.9062 - val_loss: 65211281408.0000 - val_mae: 171068.5312
Epoch 23/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 72560427008.0000 - mae:
169040.4531 - val_loss: 64209510400.0000 - val_mae: 169482.6094
Epoch 24/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 63260352512.0000 - mae:
163373.3750 - val_loss: 63336583168.0000 - val_mae: 168007.4688
Epoch 25/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 75149402112.0000 - mae:
170702.6719 - val_loss: 62443122688.0000 - val_mae: 166545.0469
Epoch 26/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 78125408256.0000 - mae:
169855.1094 - val_loss: 61638213632.0000 - val_mae: 165250.9531
Epoch 27/50
```

```
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 72557985792.0000 - mae:
167799.2656 - val_loss: 60826849280.0000 - val_mae: 164072.8594
Epoch 28/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 62515372032.0000 - mae:
161839.1562 - val_loss: 60192935936.0000 - val_mae: 162926.0625
Epoch 29/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 68945854464.0000 - mae:
163559.2188 - val_loss: 59564281856.0000 - val_mae: 161950.1406
Epoch 30/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 62408949760.0000 - mae:
159809.5000 - val_loss: 58988462080.0000 - val_mae: 161143.9531
Epoch 31/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 74693435392.0000 - mae:
165930.2344 - val_loss: 58452058112.0000 - val_mae: 159957.2031
Epoch 32/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 71271792640.0000 - mae:
164006.4375 - val_loss: 57891512320.0000 - val_mae: 159125.7812
Epoch 33/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 67738824704.0000 - mae:
159827.0000 - val_loss: 57492201472.0000 - val_mae: 158502.8438
Epoch 34/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 72449572864.0000 - mae:
166947.1875 - val_loss: 57122181120.0000 - val_mae: 157538.7344
Epoch 35/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 64835440640.0000 - mae:
162308.2969 - val_loss: 56775282688.0000 - val_mae: 156812.7031
Epoch 36/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 79214141440.0000 - mae:
164103.9844 - val_loss: 56406032384.0000 - val_mae: 156368.1875
Epoch 37/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 81792524288.0000 - mae:
163965.9688 - val_loss: 56082558976.0000 - val_mae: 155722.9531
Epoch 38/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 58844864512.0000 - mae:
157032.7500 - val_loss: 55806951424.0000 - val_mae: 155217.9844
Epoch 39/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - loss: 82327986176.0000 - mae:
168833.8438 - val_loss: 55594172416.0000 - val_mae: 154678.3750
Epoch 40/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 68388806656.0000 - mae:
160937.9062 - val_loss: 55348846592.0000 - val_mae: 154198.7656
Epoch 41/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 59182628864.0000 - mae:
157347.3906 - val_loss: 55186419712.0000 - val_mae: 153980.8906
Epoch 42/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 68994842624.0000 - mae:
157316.4531 - val_loss: 54976049152.0000 - val_mae: 153538.0469
Epoch 43/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 60257439744.0000 - mae:
```

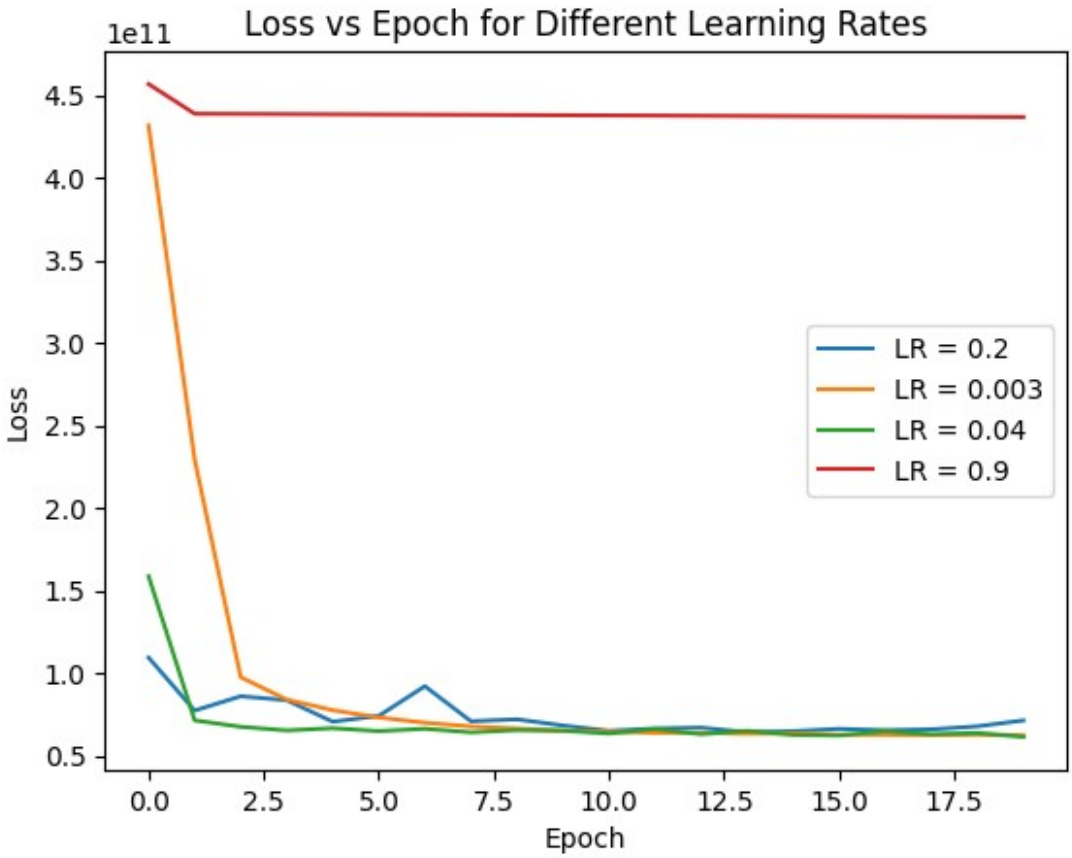
```
153893.9844 - val_loss: 54843449344.0000 - val_mae: 153294.1250
Epoch 44/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 59175518208.0000 - mae:
156518.9688 - val_loss: 54677446656.0000 - val_mae: 153118.3750
Epoch 45/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 59687677952.0000 - mae:
155208.0625 - val_loss: 54512844800.0000 - val_mae: 152568.6406
Epoch 46/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 66751307776.0000 - mae:
160858.9219 - val_loss: 54407536640.0000 - val_mae: 152395.2344
Epoch 47/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 71638532096.0000 - mae:
160795.3594 - val_loss: 54291611648.0000 - val_mae: 152327.1875
Epoch 48/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 59797135360.0000 - mae:
155533.3750 - val_loss: 54191955968.0000 - val_mae: 152035.2656
Epoch 49/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 54856982528.0000 - mae:
152417.5625 - val_loss: 54075068416.0000 - val_mae: 151957.8438
Epoch 50/50
92/92 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 63698620416.0000 - mae:
157464.9688 - val_loss: 54047567872.0000 - val_mae: 151988.6094
29/29 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 188069642240.0000 -
mae: 175904.1875
Test Loss: 985996197888.0
Test MAE: 206623.859375
```


Loss vs Epoch
1e11
4.5
4.0
3.5
3.0
2.5
2.0
1.5
1.0
0.5
Loss
0
10
20
30
40
50
Epoch
Training Loss
Validation Loss



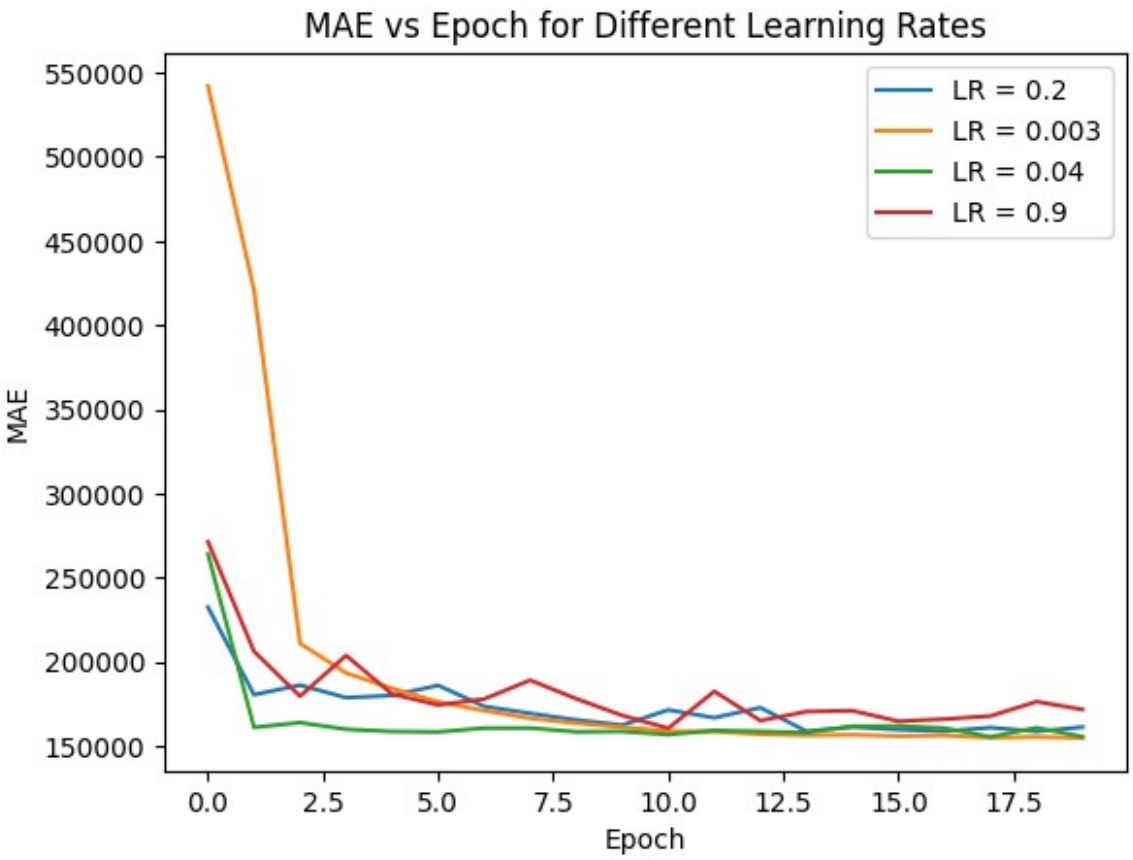

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
```

```
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```




```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
```

```
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```



Training/Testing 70/30

```python
# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

# Load Dataset
data = pd.read_csv("/content/data.csv")

print(data.head())
print(data.columns)
```

```python
# Drop unnecessary columns
data = data.drop(['date','street','city','statezip','country'],
axis=1)

# Define X and Y
X = data.drop('price', axis=1)
y = data['price']

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# Normalization
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# -------------------------------
# Main Model Training
# -------------------------------

model = Sequential()

model.add(Dense(128, activation='relu',
input_shape=(X_train.shape[1],)))
model.add(Dense(64, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(1))

optimizer = Adam(learning_rate=0.001)

model.compile(
    optimizer=optimizer,
    loss='mean_squared_error',
    metrics=['mae']
)

# Model Summary
model.summary()

# Train Model
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2
)
```

```python
# Evaluate Model
loss, mae = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test MAE:", mae)

# -------------------------------
# Visualization 1: Loss vs Epoch
# -------------------------------

plt.figure()
plt.plot(history.history['loss'], label="Training Loss")
plt.plot(history.history['val_loss'], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss vs Epoch")
plt.legend()
plt.show()

# -------------------------------
# Visualization 2: MAE vs Epoch
# -------------------------------

plt.figure()
plt.plot(history.history['mae'], label="Training MAE")
plt.plot(history.history['val_mae'], label="Validation MAE")
plt.xlabel("Epoch")
plt.ylabel("MAE")
plt.title("MAE vs Epoch")
plt.legend()
plt.show()

# -------------------------------
# Learning Rate Experiment
# -------------------------------

learning_rates = [0.2, 0.003, 0.04, 0.9]

plt.figure()

for lr in learning_rates:

    temp_model = Sequential()
    temp_model.add(Dense(128, activation='relu',
input_shape=(X_train.shape[1],)))
    temp_model.add(Dense(64, activation='relu'))
    temp_model.add(Dense(32, activation='relu'))
    temp_model.add(Dense(1))
```

```
    optimizer = Adam(learning_rate=lr)

    temp_model.compile(
        optimizer=optimizer,
        loss='mean_squared_error',
        metrics=['mae']
    )

    history_lr = temp_model.fit(
        X_train,
        y_train,
        epochs=20,
        batch_size=32,
        verbose=0
    )

    plt.plot(history_lr.history['loss'], label=f"LR = {lr}")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss vs Epoch for Different Learning Rates")
plt.legend()
plt.show()

# -------------------------------
# MAE vs Epoch for Learning Rates
# -------------------------------

plt.figure()

for lr in learning_rates:

    temp_model = Sequential()
    temp_model.add(Dense(128, activation='relu',
input_shape=(X_train.shape[1],)))
    temp_model.add(Dense(64, activation='relu'))
    temp_model.add(Dense(32, activation='relu'))
    temp_model.add(Dense(1))

    optimizer = Adam(learning_rate=lr)

    temp_model.compile(
        optimizer=optimizer,
        loss='mean_squared_error',
        metrics=['mae']
    )

    history_lr = temp_model.fit(
        X_train,
        y_train,
```

```
        epochs=20,
        batch_size=32,
        verbose=0
    )

    plt.plot(history_lr.history['mae'], label=f"LR = {lr}")

plt.xlabel("Epoch")
plt.ylabel("MAE")
plt.title("MAE vs Epoch for Different Learning Rates")
plt.legend()
plt.show()
```

```
                  date      price  bedrooms  bathrooms  sqft_living
sqft_lot  \
0  2014-05-02 00:00:00   313000.0       3.0       1.50         1340
7912
1  2014-05-02 00:00:00  2384000.0       5.0       2.50         3650
9050
2  2014-05-02 00:00:00   342000.0       3.0       2.00         1930
11947
3  2014-05-02 00:00:00   420000.0       3.0       2.25         2000
8030
4  2014-05-02 00:00:00   550000.0       4.0       2.50         1940
10500

   floors  waterfront  view  condition  sqft_above  sqft_basement
yr_built  \
0     1.5           0     0          3        1340              0
1955
1     2.0           0     4          5        3370            280
1921
2     1.0           0     0          4        1930              0
1966
3     1.0           0     0          4        1000           1000
1963
4     1.0           0     0          4        1140            800
1976

   yr_renovated                     street       city  statezip country

0          2005      18810 Densmore Ave N  Shoreline  WA 98133     USA

1             0           709 W Blaine St    Seattle  WA 98119     USA

2             0  26206-26214 143rd Ave SE       Kent  WA 98042     USA

3             0           857 170th Pl NE   Bellevue  WA 98008     USA

4          1992         9105 170th Ave NE    Redmond  WA 98052     USA
```

```
Index(['date', 'price', 'bedrooms', 'bathrooms', 'sqft_living',
'sqft_lot',
       'floors', 'waterfront', 'view', 'condition', 'sqft_above',
       'sqft_basement', 'yr_built', 'yr_renovated', 'street', 'city',
       'statezip', 'country'],
      dtype='object')
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```

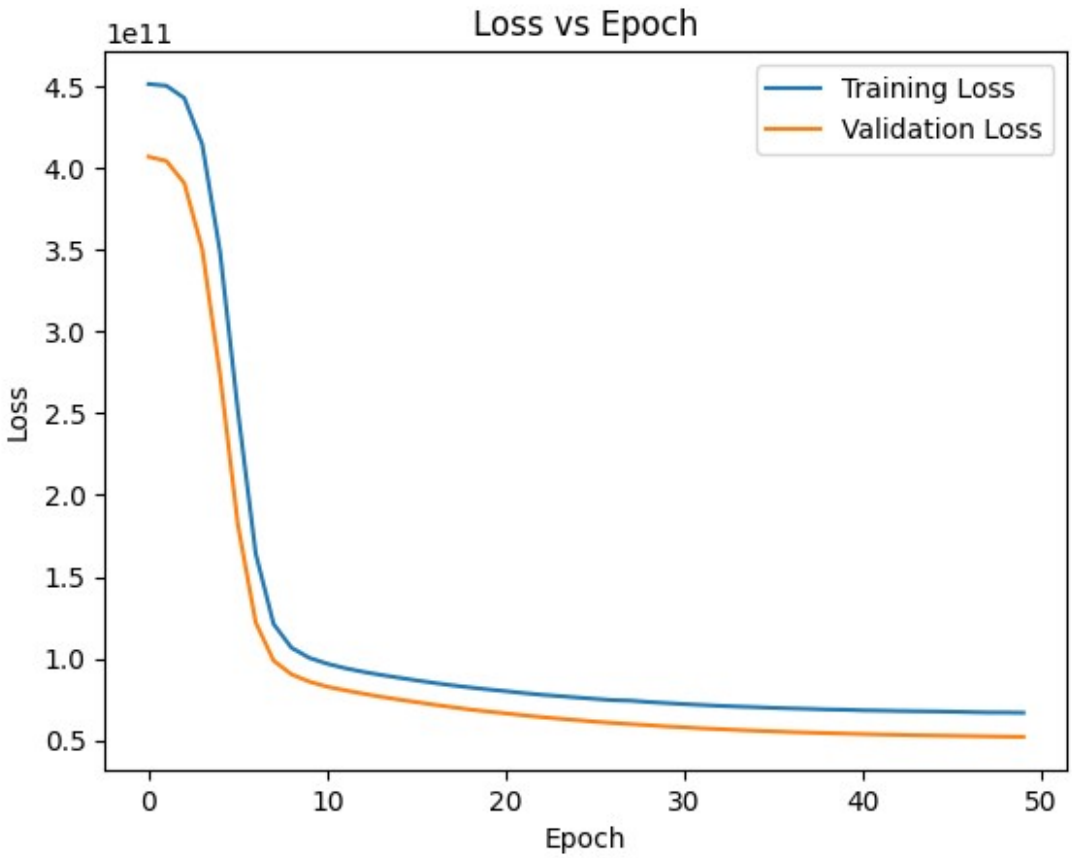
```
Model: "sequential_18"

┏━━━
```

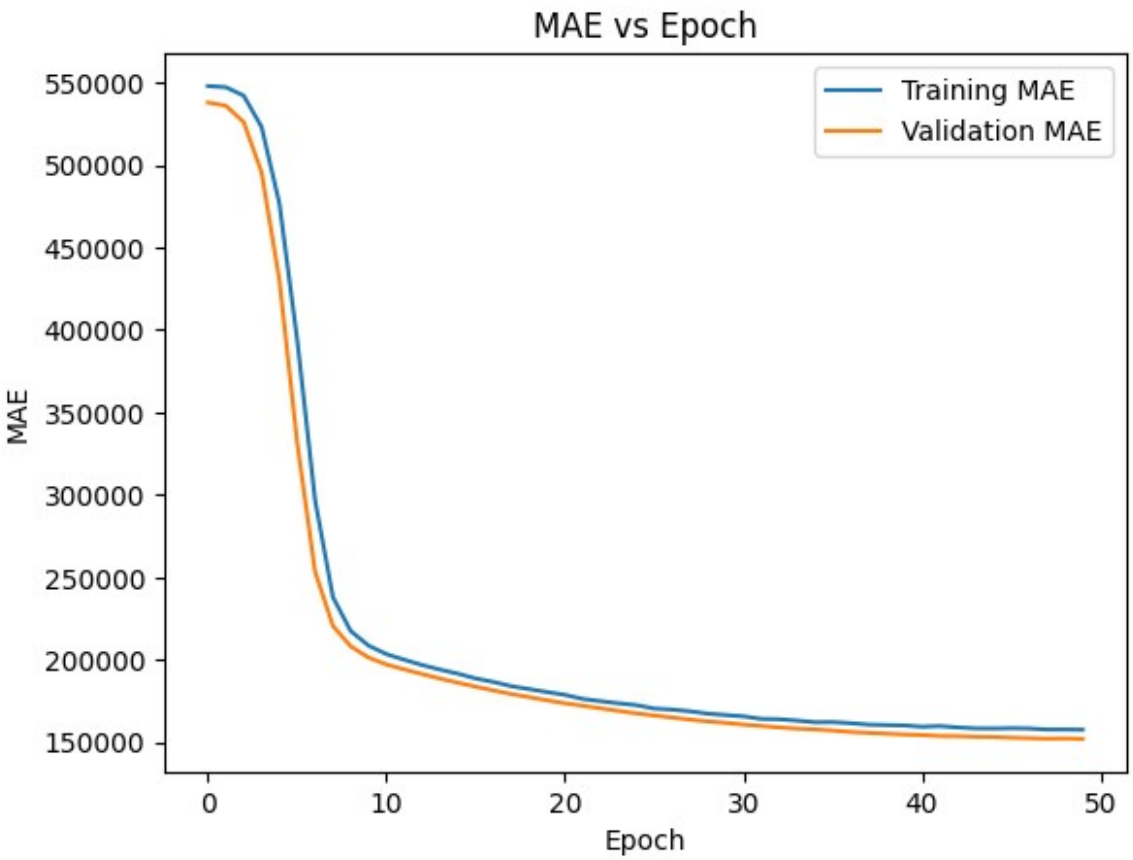
```
mae: 538000.6875 - val_loss: 404036255744.0000 - val_mae: 535878.5625
Epoch 3/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 480076791808.0000 -
mae: 553744.1875 - val_loss: 390528401408.0000 - val_mae: 525955.1875
Epoch 4/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 433943576576.0000 -
mae: 538197.3125 - val_loss: 350374526976.0000 - val_mae: 495592.6562
Epoch 5/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 331027939328.0000 -
mae: 479682.3750 - val_loss: 273689296896.0000 - val_mae: 430281.2500
Epoch 6/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 268834308096.0000 -
mae: 421418.8125 - val_loss: 181763719168.0000 - val_mae: 331470.3750
Epoch 7/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 173748682752.0000 -
mae: 321955.0625 - val_loss: 122034692096.0000 - val_mae: 253443.8438
Epoch 8/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 135039975424.0000 -
mae: 250928.4375 - val_loss: 98742968320.0000 - val_mae: 220552.2188
Epoch 9/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 104212897792.0000 -
mae: 219794.5312 - val_loss: 90406002688.0000 - val_mae: 208144.7500
Epoch 10/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 115305824256.0000 -
mae: 212458.8750 - val_loss: 85891686400.0000 - val_mae: 201329.6719
Epoch 11/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 97914511360.0000 - mae:
202606.4688 - val_loss: 82819473408.0000 - val_mae: 197205.7031
Epoch 12/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 90579787776.0000 - mae:
195131.7656 - val_loss: 80605290496.0000 - val_mae: 194182.3125
Epoch 13/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 88953970688.0000 - mae:
196221.9844 - val_loss: 78594252800.0000 - val_mae: 191337.8906
Epoch 14/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 91092303872.0000 - mae:
197739.1719 - val_loss: 76707160064.0000 - val_mae: 188644.8281
Epoch 15/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 95225462784.0000 - mae:
194245.3906 - val_loss: 74992361472.0000 - val_mae: 186096.4062
Epoch 16/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 78230052864.0000 - mae:
183957.0000 - val_loss: 73415909376.0000 - val_mae: 183710.5625
Epoch 17/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 84273217536.0000 - mae:
184919.0000 - val_loss: 71783063552.0000 - val_mae: 181430.5156
Epoch 18/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 69116837888.0000 - mae:
178126.0625 - val_loss: 70332932096.0000 - val_mae: 179265.2188
```

```
Epoch 19/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 84602716160.0000 - mae:
183524.4531 - val_loss: 68863877120.0000 - val_mae: 177366.9531
Epoch 20/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 91330453504.0000 - mae:
184929.3125 - val_loss: 67649662976.0000 - val_mae: 175459.8438
Epoch 21/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 96447184896.0000 - mae:
188496.4219 - val_loss: 66518523904.0000 - val_mae: 173668.4062
Epoch 22/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 92884402176.0000 - mae:
181031.9844 - val_loss: 65321791488.0000 - val_mae: 172158.5469
Epoch 23/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 69033934848.0000 - mae:
172351.1094 - val_loss: 64252043264.0000 - val_mae: 170615.8438
Epoch 24/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 92417654784.0000 - mae:
183767.6250 - val_loss: 63270404096.0000 - val_mae: 169038.5000
Epoch 25/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 100281286656.0000 -
mae: 179603.5156 - val_loss: 62397898752.0000 - val_mae: 167563.8125
Epoch 26/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 79832784896.0000 - mae:
176221.5312 - val_loss: 61455470592.0000 - val_mae: 166269.5938
Epoch 27/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 68548227072.0000 - mae:
166556.7812 - val_loss: 60662173696.0000 - val_mae: 165029.9844
Epoch 28/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 75336540160.0000 - mae:
165546.7656 - val_loss: 59971837952.0000 - val_mae: 163770.5781
Epoch 29/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 75852185600.0000 - mae:
169054.2500 - val_loss: 59294785536.0000 - val_mae: 162674.3906
Epoch 30/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 79801393152.0000 - mae:
173237.1875 - val_loss: 58548551680.0000 - val_mae: 161812.8750
Epoch 31/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 62375673856.0000 - mae:
158228.9844 - val_loss: 58020212736.0000 - val_mae: 160736.5312
Epoch 32/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 66099474432.0000 - mae:
161535.5156 - val_loss: 57351413760.0000 - val_mae: 159963.8281
Epoch 33/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 70678216704.0000 - mae:
160527.5625 - val_loss: 56891707392.0000 - val_mae: 159025.5156
Epoch 34/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 67930079232.0000 - mae:
160719.7031 - val_loss: 56386445312.0000 - val_mae: 158323.0781
Epoch 35/50
```

```
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 62373675008.0000 - mae:
159038.0000 - val_loss: 55908052992.0000 - val_mae: 157858.4844
Epoch 36/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 69823168512.0000 - mae:
160051.2031 - val_loss: 55514349568.0000 - val_mae: 157161.6094
Epoch 37/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 72006189056.0000 - mae:
163542.4062 - val_loss: 55090438144.0000 - val_mae: 156295.9062
Epoch 38/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 71738220544.0000 - mae:
164648.5469 - val_loss: 54768672768.0000 - val_mae: 155749.0938
Epoch 39/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 63079890944.0000 - mae:
161785.7656 - val_loss: 54464905216.0000 - val_mae: 155247.0000
Epoch 40/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 72303902720.0000 - mae:
158099.3594 - val_loss: 54169165824.0000 - val_mae: 154728.0312
Epoch 41/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 64645931008.0000 - mae:
160272.4531 - val_loss: 53908520960.0000 - val_mae: 154415.6719
Epoch 42/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 65920729088.0000 - mae:
159692.9219 - val_loss: 53623611392.0000 - val_mae: 153852.3125
Epoch 43/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 58411241472.0000 - mae:
158091.1406 - val_loss: 53403779072.0000 - val_mae: 153738.5625
Epoch 44/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 69696872448.0000 - mae:
162859.9062 - val_loss: 53142560768.0000 - val_mae: 153375.2188
Epoch 45/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 62887522304.0000 - mae:
158744.0781 - val_loss: 52976181248.0000 - val_mae: 153252.3125
Epoch 46/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 70602817536.0000 - mae:
159758.4688 - val_loss: 52819181568.0000 - val_mae: 152774.0312
Epoch 47/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 76904980480.0000 - mae:
160112.4219 - val_loss: 52637106176.0000 - val_mae: 152523.2188
Epoch 48/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 65835831296.0000 - mae:
159790.0938 - val_loss: 52471259136.0000 - val_mae: 152213.9062
Epoch 49/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 59313184768.0000 - mae:
153479.8750 - val_loss: 52313477120.0000 - val_mae: 152317.8281
Epoch 50/50
81/81 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 66575073280.0000 - mae:
155943.0938 - val_loss: 52181880832.0000 - val_mae: 152117.0000
44/44 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 383181717504.0000 -
mae: 183092.9531
```

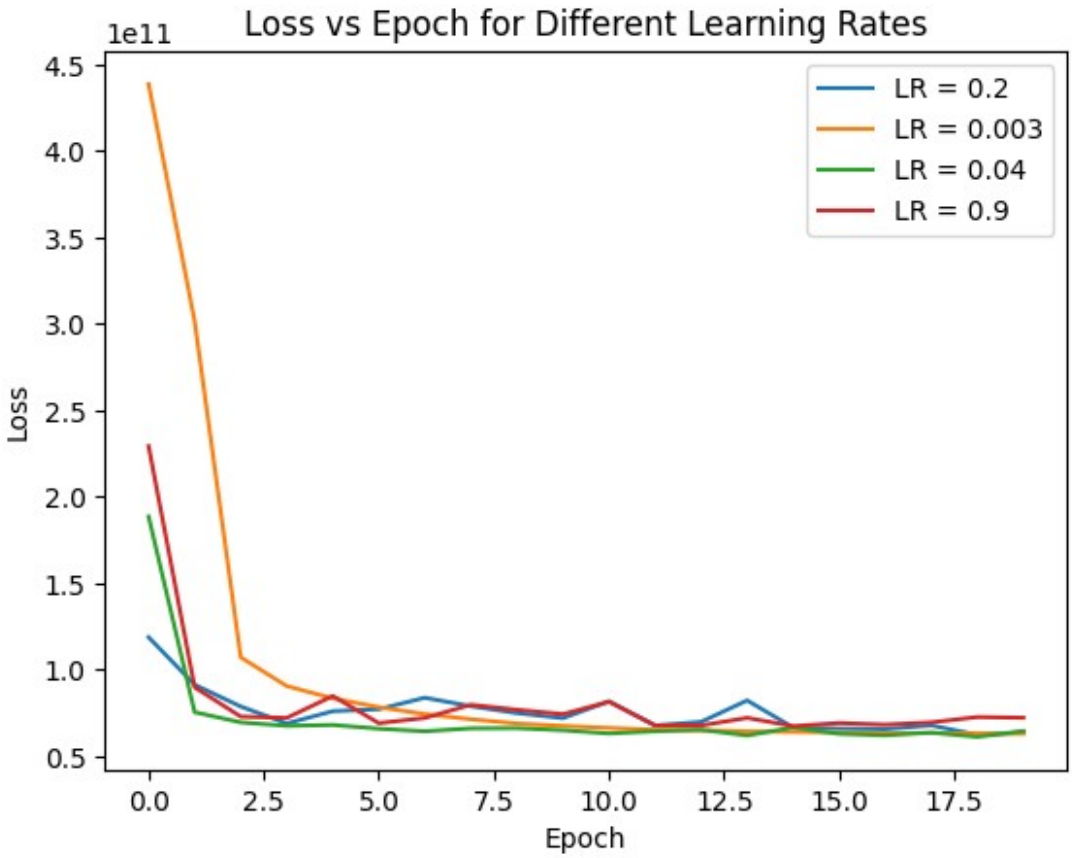
```
Test Loss: 678150668288.0
Test MAE: 191310.96875
```





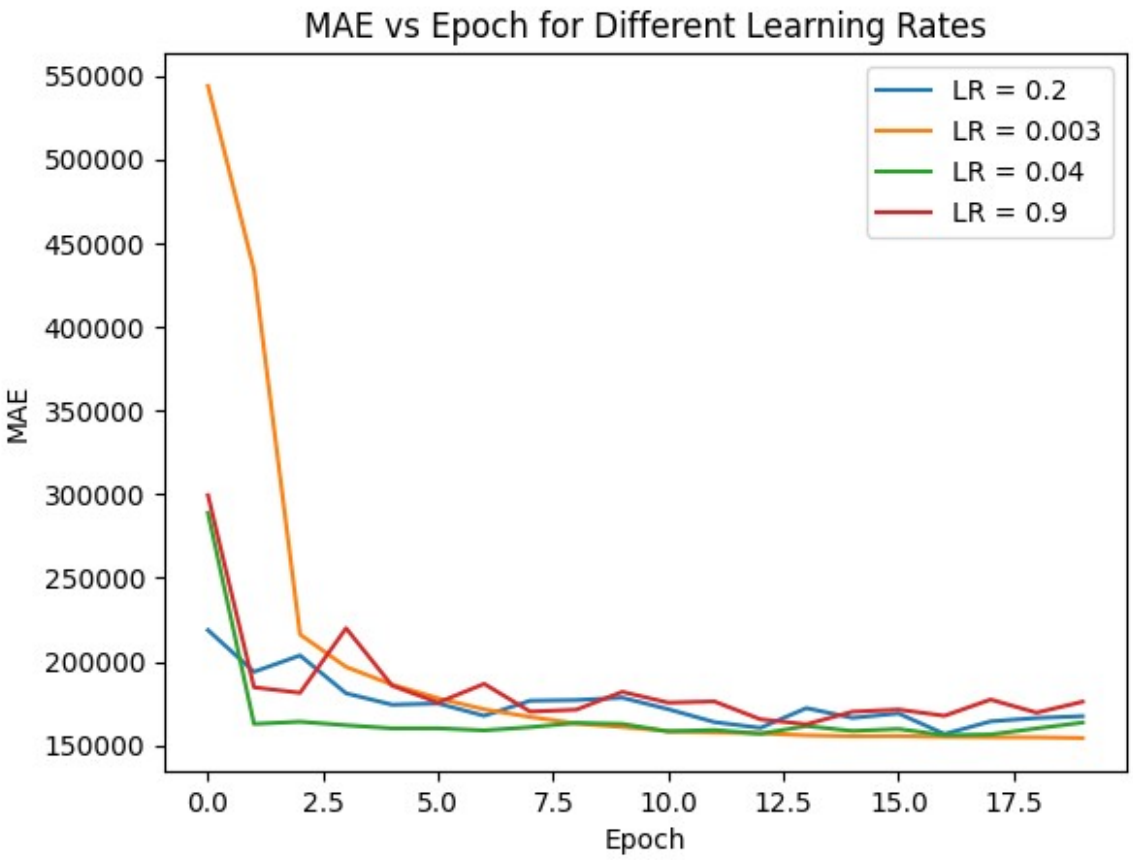

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
```

```
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```




```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
```

```
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```



Training/Testing 90/10

```
# Import Libraries
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense
from tensorflow.keras.optimizers import Adam

# Load Dataset
data = pd.read_csv("/content/data.csv")

print(data.head())
print(data.columns)
```

```python
# Drop unnecessary columns
data = data.drop(['date','street','city','statezip','country'],
axis=1)

# Define X and Y
X = data.drop('price', axis=1)
y = data['price']

# Train Test Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.1, random_state=42
)

# Normalization
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# -------------------------------
# Main Model Training
# -------------------------------

model = Sequential()

model.add(Dense(128, activation='relu',
input_shape=(X_train.shape[1],)))
model.add(Dense(64, activation='relu'))
model.add(Dense(32, activation='relu'))
model.add(Dense(1))

optimizer = Adam(learning_rate=0.001)

model.compile(
    optimizer=optimizer,
    loss='mean_squared_error',
    metrics=['mae']
)

# Model Summary
model.summary()

# Train Model
history = model.fit(
    X_train,
    y_train,
    epochs=50,
    batch_size=32,
    validation_split=0.2
)
```

```python
# Evaluate Model
loss, mae = model.evaluate(X_test, y_test)

print("Test Loss:", loss)
print("Test MAE:", mae)

# -------------------------------
# Visualization 1: Loss vs Epoch
# -------------------------------

plt.figure()
plt.plot(history.history['loss'], label="Training Loss")
plt.plot(history.history['val_loss'], label="Validation Loss")
plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss vs Epoch")
plt.legend()
plt.show()

# -------------------------------
# Visualization 2: MAE vs Epoch
# -------------------------------

plt.figure()
plt.plot(history.history['mae'], label="Training MAE")
plt.plot(history.history['val_mae'], label="Validation MAE")
plt.xlabel("Epoch")
plt.ylabel("MAE")
plt.title("MAE vs Epoch")
plt.legend()
plt.show()

# -------------------------------
# Learning Rate Experiment
# -------------------------------

learning_rates = [0.2, 0.003, 0.04, 0.9]

plt.figure()

for lr in learning_rates:

    temp_model = Sequential()
    temp_model.add(Dense(128, activation='relu',
input_shape=(X_train.shape[1],)))
    temp_model.add(Dense(64, activation='relu'))
    temp_model.add(Dense(32, activation='relu'))
    temp_model.add(Dense(1))
```

```
    optimizer = Adam(learning_rate=lr)

    temp_model.compile(
        optimizer=optimizer,
        loss='mean_squared_error',
        metrics=['mae']
    )

    history_lr = temp_model.fit(
        X_train,
        y_train,
        epochs=20,
        batch_size=32,
        verbose=0
    )

    plt.plot(history_lr.history['loss'], label=f"LR = {lr}")

plt.xlabel("Epoch")
plt.ylabel("Loss")
plt.title("Loss vs Epoch for Different Learning Rates")
plt.legend()
plt.show()

# -------------------------------
# MAE vs Epoch for Learning Rates
# -------------------------------

plt.figure()

for lr in learning_rates:

    temp_model = Sequential()
    temp_model.add(Dense(128, activation='relu',
input_shape=(X_train.shape[1],)))
    temp_model.add(Dense(64, activation='relu'))
    temp_model.add(Dense(32, activation='relu'))
    temp_model.add(Dense(1))

    optimizer = Adam(learning_rate=lr)

    temp_model.compile(
        optimizer=optimizer,
        loss='mean_squared_error',
        metrics=['mae']
    )

    history_lr = temp_model.fit(
        X_train,
        y_train,
```

```
        epochs=20,
        batch_size=32,
        verbose=0
    )

    plt.plot(history_lr.history['mae'], label=f"LR = {lr}")

plt.xlabel("Epoch")
plt.ylabel("MAE")
plt.title("MAE vs Epoch for Different Learning Rates")
plt.legend()
plt.show()
```

```
                  date      price  bedrooms  bathrooms  sqft_living
sqft_lot  \
0  2014-05-02 00:00:00   313000.0       3.0       1.50         1340
7912
1  2014-05-02 00:00:00  2384000.0       5.0       2.50         3650
9050
2  2014-05-02 00:00:00   342000.0       3.0       2.00         1930
11947
3  2014-05-02 00:00:00   420000.0       3.0       2.25         2000
8030
4  2014-05-02 00:00:00   550000.0       4.0       2.50         1940
10500

   floors  waterfront  view  condition  sqft_above  sqft_basement
yr_built  \
0     1.5           0     0          3        1340              0
1955
1     2.0           0     4          5        3370            280
1921
2     1.0           0     0          4        1930              0
1966
3     1.0           0     0          4        1000           1000
1963
4     1.0           0     0          4        1140            800
1976

   yr_renovated                     street       city  statezip country

0          2005      18810 Densmore Ave N  Shoreline  WA 98133     USA

1             0           709 W Blaine St    Seattle  WA 98119     USA

2             0  26206-26214 143rd Ave SE       Kent  WA 98042     USA

3             0           857 170th Pl NE   Bellevue  WA 98008     USA

4          1992         9105 170th Ave NE    Redmond  WA 98052     USA
```

```
Index(['date', 'price', 'bedrooms', 'bathrooms', 'sqft_living',
'sqft_lot',
       'floors', 'waterfront', 'view', 'condition', 'sqft_above',
       'sqft_basement', 'yr_built', 'yr_renovated', 'street', 'city',
       'statezip', 'country'],
      dtype='object')
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```

```
Model: "sequential_27"
```

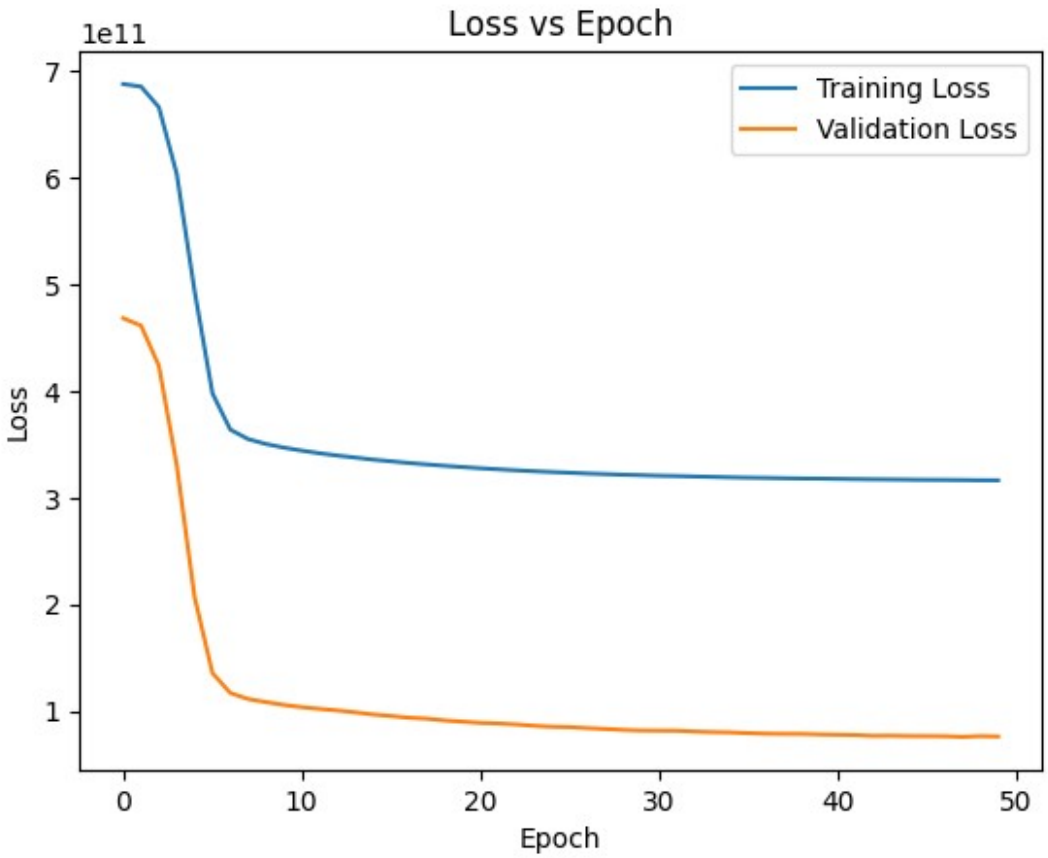
```
┏━━━
```

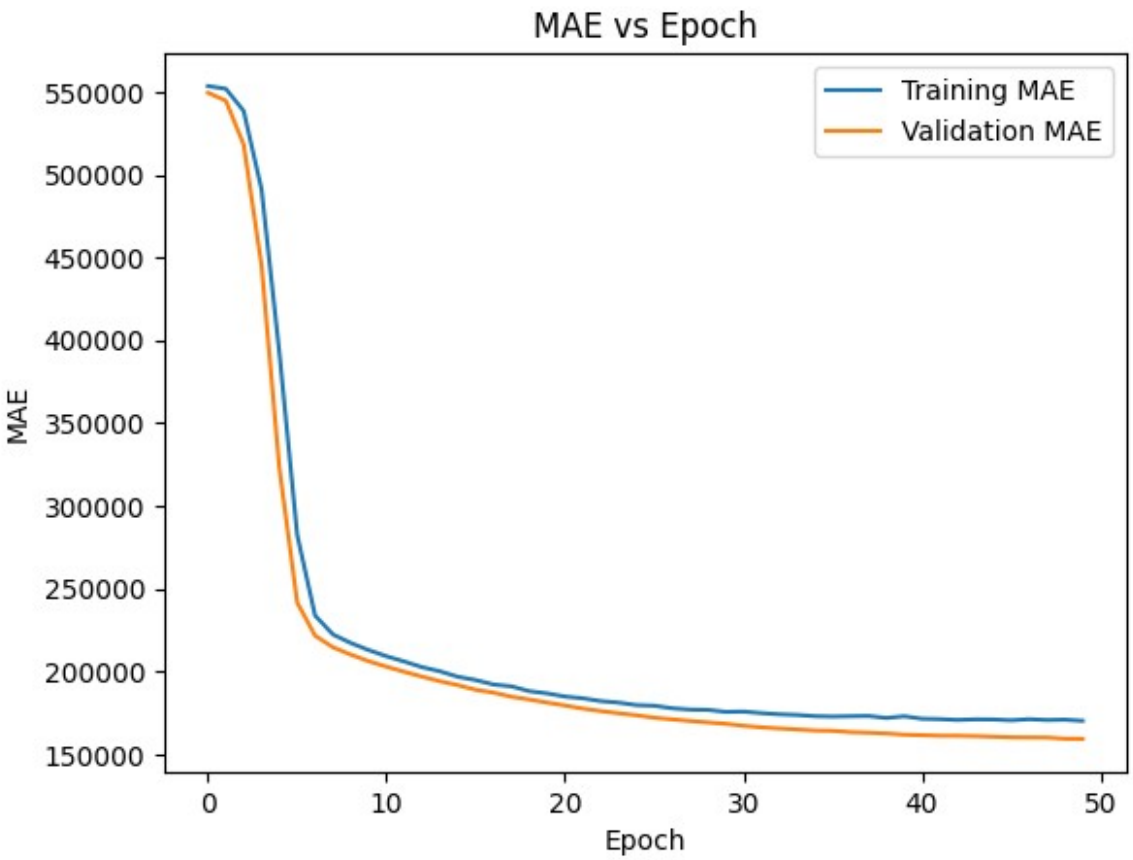
```
mae: 550641.0000 - val_loss: 461568933888.0000 - val_mae: 544605.4375
Epoch 3/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 606374658048.0000 -
mae: 543051.0625 - val_loss: 423798243328.0000 - val_mae: 518103.1562
Epoch 4/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 939467866112.0000 -
mae: 523740.7188 - val_loss: 330572496896.0000 - val_mae: 445203.7812
Epoch 5/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 610179153920.0000 -
mae: 423781.9375 - val_loss: 206800437248.0000 - val_mae: 322712.2500
Epoch 6/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 325663588352.0000 -
mae: 294109.4688 - val_loss: 135474151424.0000 - val_mae: 241514.4375
Epoch 7/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 381829709824.0000 -
mae: 245744.1094 - val_loss: 116804681728.0000 - val_mae: 221613.9844
Epoch 8/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 317110910976.0000 -
mae: 224527.7031 - val_loss: 111140659200.0000 - val_mae: 214571.1719
Epoch 9/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 987794178048.0000 -
mae: 238510.0469 - val_loss: 108277587968.0000 - val_mae: 210144.9844
Epoch 10/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 238241693696.0000 -
mae: 216511.7031 - val_loss: 105589006336.0000 - val_mae: 206163.4062
Epoch 11/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 306061049856.0000 -
mae: 206793.2969 - val_loss: 103505420288.0000 - val_mae: 202885.7188
Epoch 12/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 182677880832.0000 -
mae: 203041.7344 - val_loss: 101886713856.0000 - val_mae: 199752.5938
Epoch 13/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 257471070208.0000 -
mae: 207031.6094 - val_loss: 100411719680.0000 - val_mae: 196825.7344
Epoch 14/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 419259252736.0000 -
mae: 201082.8906 - val_loss: 98604367872.0000 - val_mae: 194056.7969
Epoch 15/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 182921134080.0000 -
mae: 184469.7188 - val_loss: 96624041984.0000 - val_mae: 191693.4688
Epoch 16/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 246057009152.0000 -
mae: 197536.7656 - val_loss: 95226740736.0000 - val_mae: 188893.2031
Epoch 17/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 174300790784.0000 -
mae: 189052.2812 - val_loss: 93573537792.0000 - val_mae: 187189.1406
Epoch 18/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 488592015360.0000 -
mae: 194930.9531 - val_loss: 92717441024.0000 - val_mae: 184737.3594
```

```
Epoch 19/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 179622969344.0000 -
mae: 184544.9062 - val_loss: 91057045504.0000 - val_mae: 183040.8438
Epoch 20/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 4ms/step - loss: 240381935616.0000 -
mae: 180409.2031 - val_loss: 89896402944.0000 - val_mae: 181177.4688
Epoch 21/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 261313085440.0000 -
mae: 189547.3281 - val_loss: 88776597504.0000 - val_mae: 179425.5781
Epoch 22/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 324369448960.0000 -
mae: 182377.1250 - val_loss: 88211791872.0000 - val_mae: 177561.6250
Epoch 23/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 289893449728.0000 -
mae: 185601.4375 - val_loss: 87292248064.0000 - val_mae: 176061.0312
Epoch 24/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 701397270528.0000 -
mae: 195886.6875 - val_loss: 86036897792.0000 - val_mae: 174728.7188
Epoch 25/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - loss: 260734304256.0000 -
mae: 173504.0469 - val_loss: 85171331072.0000 - val_mae: 173514.6406
Epoch 26/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 509322362880.0000 -
mae: 189409.8750 - val_loss: 84778835968.0000 - val_mae: 172030.6875
Epoch 27/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 172929499136.0000 -
mae: 172634.7500 - val_loss: 83830661120.0000 - val_mae: 171056.2500
Epoch 28/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 323864264704.0000 -
mae: 171382.3125 - val_loss: 83063349248.0000 - val_mae: 170079.7344
Epoch 29/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 268035112960.0000 -
mae: 176017.9375 - val_loss: 82185871360.0000 - val_mae: 169211.8281
Epoch 30/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 193272528896.0000 -
mae: 163139.1406 - val_loss: 81626963968.0000 - val_mae: 168381.0000
Epoch 31/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 267225153536.0000 -
mae: 178526.9844 - val_loss: 81473454080.0000 - val_mae: 167184.4375
Epoch 32/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 559212462080.0000 -
mae: 181629.1094 - val_loss: 81374584832.0000 - val_mae: 166248.0312
Epoch 33/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 355462086656.0000 -
mae: 181280.1562 - val_loss: 80680951808.0000 - val_mae: 165575.7812
Epoch 34/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 399035826176.0000 -
mae: 176558.8281 - val_loss: 80106266624.0000 - val_mae: 164974.2344
Epoch 35/50
```

```
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 221472407552.0000 -
mae: 168296.4531 - val_loss: 79832965120.0000 - val_mae: 164313.1562
Epoch 36/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - loss: 316882288640.0000 -
mae: 170725.7188 - val_loss: 79132745728.0000 - val_mae: 164131.0312
Epoch 37/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 324137189376.0000 -
mae: 173829.2969 - val_loss: 78647648256.0000 - val_mae: 163335.0469
Epoch 38/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - loss: 471267180544.0000 -
mae: 181308.6719 - val_loss: 78473576448.0000 - val_mae: 163008.1562
Epoch 39/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 374667182080.0000 -
mae: 169733.3281 - val_loss: 78430797824.0000 - val_mae: 162524.3906
Epoch 40/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 393503670272.0000 -
mae: 176648.9844 - val_loss: 77923246080.0000 - val_mae: 161795.5781
Epoch 41/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 1179870822400.0000 -
mae: 206455.6094 - val_loss: 77637189632.0000 - val_mae: 161496.8281
Epoch 42/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 442035568640.0000 -
mae: 172069.3438 - val_loss: 77374922752.0000 - val_mae: 161173.6406
Epoch 43/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 110859526144.0000 -
mae: 160447.9375 - val_loss: 76579291136.0000 - val_mae: 161148.3438
Epoch 44/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 165257822208.0000 -
mae: 163221.4531 - val_loss: 76746244096.0000 - val_mae: 160950.0938
Epoch 45/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 120970764288.0000 -
mae: 158697.4531 - val_loss: 76417261568.0000 - val_mae: 160578.8438
Epoch 46/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 94898028544.0000 -
mae: 158494.8906 - val_loss: 76359860224.0000 - val_mae: 160223.1562
Epoch 47/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 314409156608.0000 -
mae: 173139.3906 - val_loss: 76243755008.0000 - val_mae: 160127.6875
Epoch 48/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 538279182336.0000 -
mae: 170594.1719 - val_loss: 75665637376.0000 - val_mae: 160087.6875
Epoch 49/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 492217106432.0000 -
mae: 174930.0938 - val_loss: 76327124992.0000 - val_mae: 159334.8281
Epoch 50/50
104/104 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 210297323520.0000 -
mae: 162670.9375 - val_loss: 75993022464.0000 - val_mae: 159291.1094
15/15 ━━━━━━━━━━━━━━━━━━━━ 0s 5ms/step - loss: 70428352512.0000 - mae:
177843.8438
```

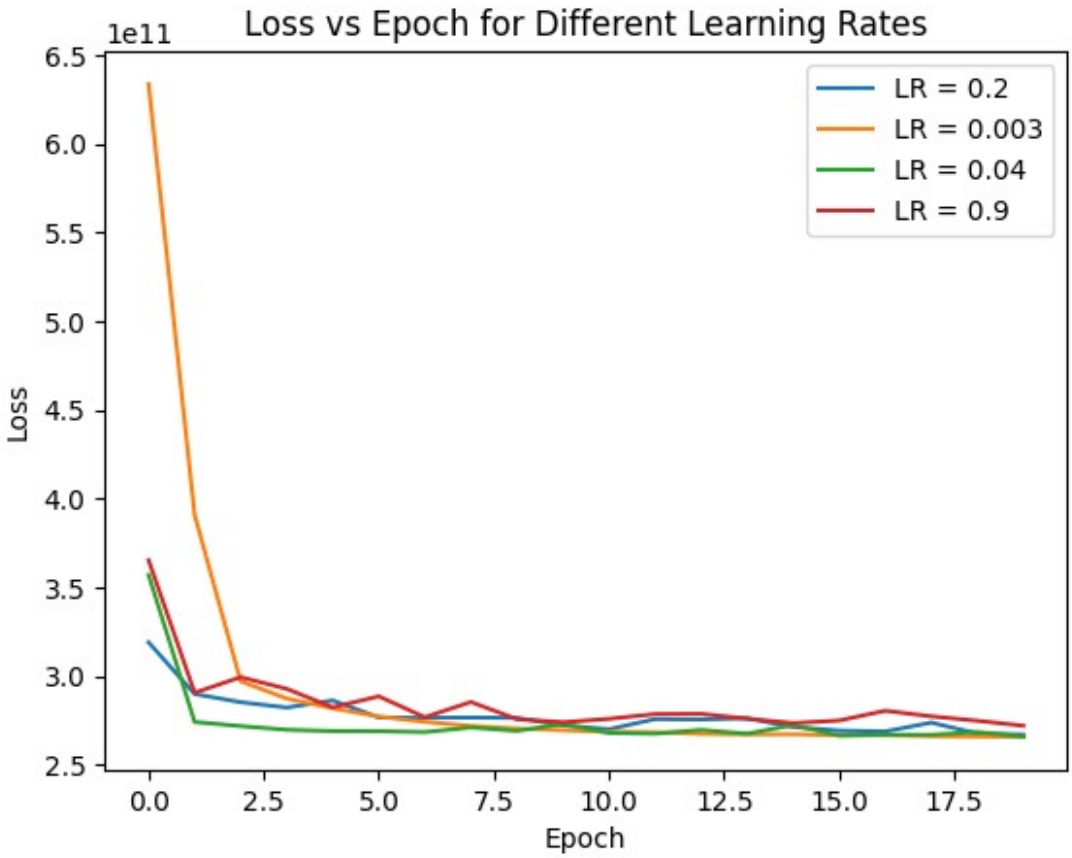
```
Test Loss: 60824940544.0
Test MAE: 164246.296875
```

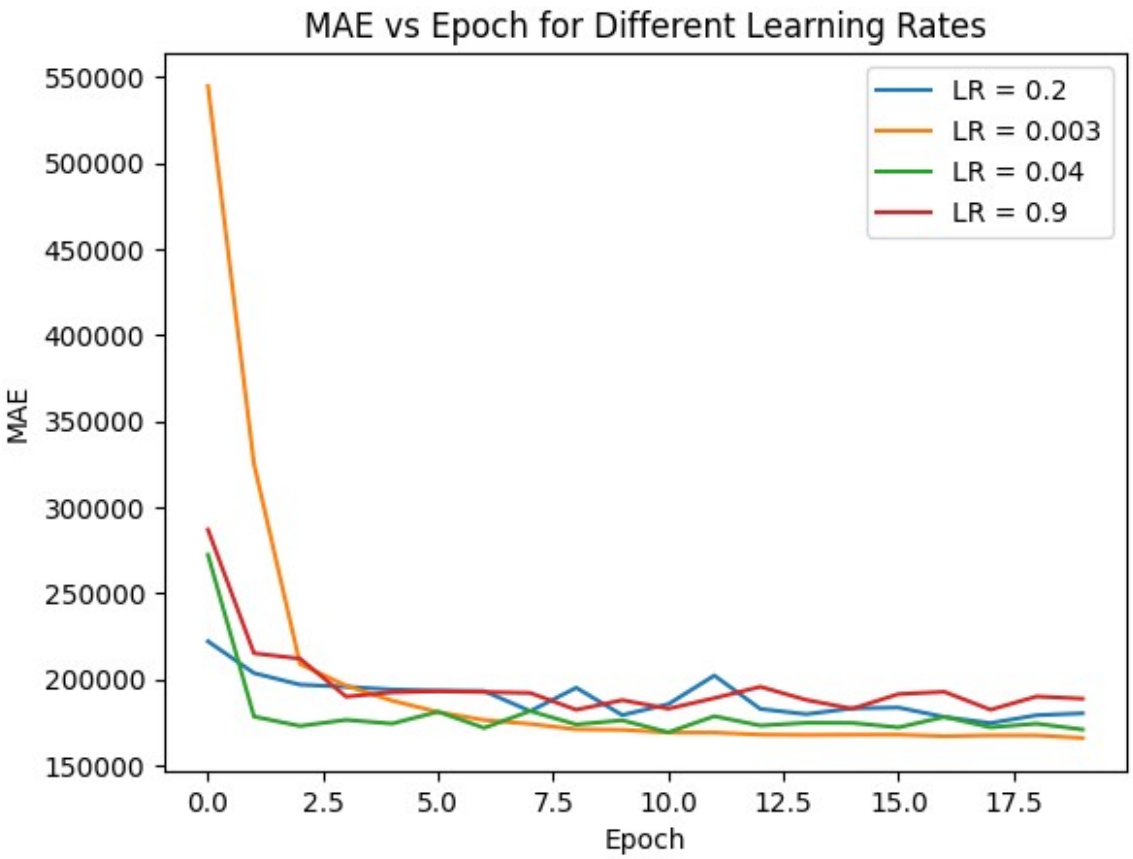





```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
```

```
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```




```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/
dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
argument to a layer. When using Sequential models, prefer using an
`Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py
```

```
:93: UserWarning: Do not pass an `input_shape`/`input_dim` argument to
a layer. When using Sequential models, prefer using an `Input(shape)`
object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer,
**kwargs)
```